Executive Summary

ExampleCorp's knowledge is located in three sources that are not connected to each other. The technical documentation at docs.examplecorp.com, the OpenAPI interfaces and the source code. Each of these sources was created for a different audience and maintained by different teams.

The problem with having three sources is that they are not in sync. For example, when an API gets updated the OpenAPI spec reflects the change immediately but the documentation and code examples lag behind. Users spend time going between all three sources trying to determine what the latest information is. Support teams get unnecessary questions because the documentation is inconsistent with the spec. Teams now have the ongoing task of trying to keep everything synchronized as the platform changes.

The Unified Knowledge Layer (UKL) is a proposal for taking all three sources and unifying them into a single queryable system. The UKL is a knowledge graph where the content from documentation, OpenAPI and code is reduced to individual entities and linked through defined relationships. As an example an endpoint like POST /transfers would be represented as its own entity in the graph and linked to the documentation section that explains how to use it.

Instead of having to manually searching three sources the UKL can do the same from one query. To find information on how to create a balance account transfer the results will include documentation, endpoint definition and the code examples because they are linked in the graph. Support teams can benefit too since common questions already have the OpenAPI definitions and documentation connected.

The other thing the graph gives us is detecting when sources are out of sync. If an endpoint gets a new parameter in the spec but the documentation has not been updated the system flags it. Right now that kind of problem gets caught when a user runs into it. With the UKL the detection is automated and the team is notified before it reaches the end user.

Conceptual Model

The Conceptual Model defines how knowledge is represented in the UKL. The three sources of documentation, OpenAPI and code are mapped into a unified structure where APIs, endpoints, schemas, doc sections and code examples become entities that can be queried.

Core Entities

The UKL is built from a set of core entities derived from the APIs. The table below defines each entity type, where the data comes from and the key attributes.

Entity Definitions

Entity	Definition	Source(s)	Key Attributes
API	A logical grouping of endpoints that expose related ExampleCorp functionality (Balance Platform API, Disputes API, Transaction Webhooks)	OpenAPI, Docs	name, version, baseURL, description, specVersion

APIEndpoint	A specific operation exposed by an API, identified by path and HTTP method (POST /balancePlatform.transaction.created)	OpenAPI, Code	path, method, operationId, summary, deprecated, addedInVersion
SchemaElement	A field, object, or data type defined in OpenAPI specifications (Amount, Transaction, BalanceAccount)	OpenAPI	name, type, required, description, constraints, format
DocSection	A unit of human-authored documentation such as a guide, tutorial, or reference page ("How to create a payment session")	Git/CMS	title, content, sourceFile, commitHash, docType, lastUpdated
CodeSnippet	A fragment of source code that demonstrates or implements an API operation (Java SDK example for creating a transfer)	Code Repos	language, content, filePath, functionName, startLine, endLine
IntegrationFlow	A sequence of API calls and steps representing a complete developer workflow ("Authentication Flow")	Docs, OpenAPI	name, steps, servicesInvolved, authFlow, errorHandling
Concept	A domain term or business entity introduced in documentation or code (balancePlatform, merchantAccount, paymentInstrument)	Docs, Code, OpenAPI	name, definition, aliases, relatedConcepts
WebhookEvent	A notification sent by ExampleCorp when a specific event occurs (balancePlatform.transaction.created)	OpenAPI Webhooks	eventType, triggerCondition, payloadSchema, securityRequirements

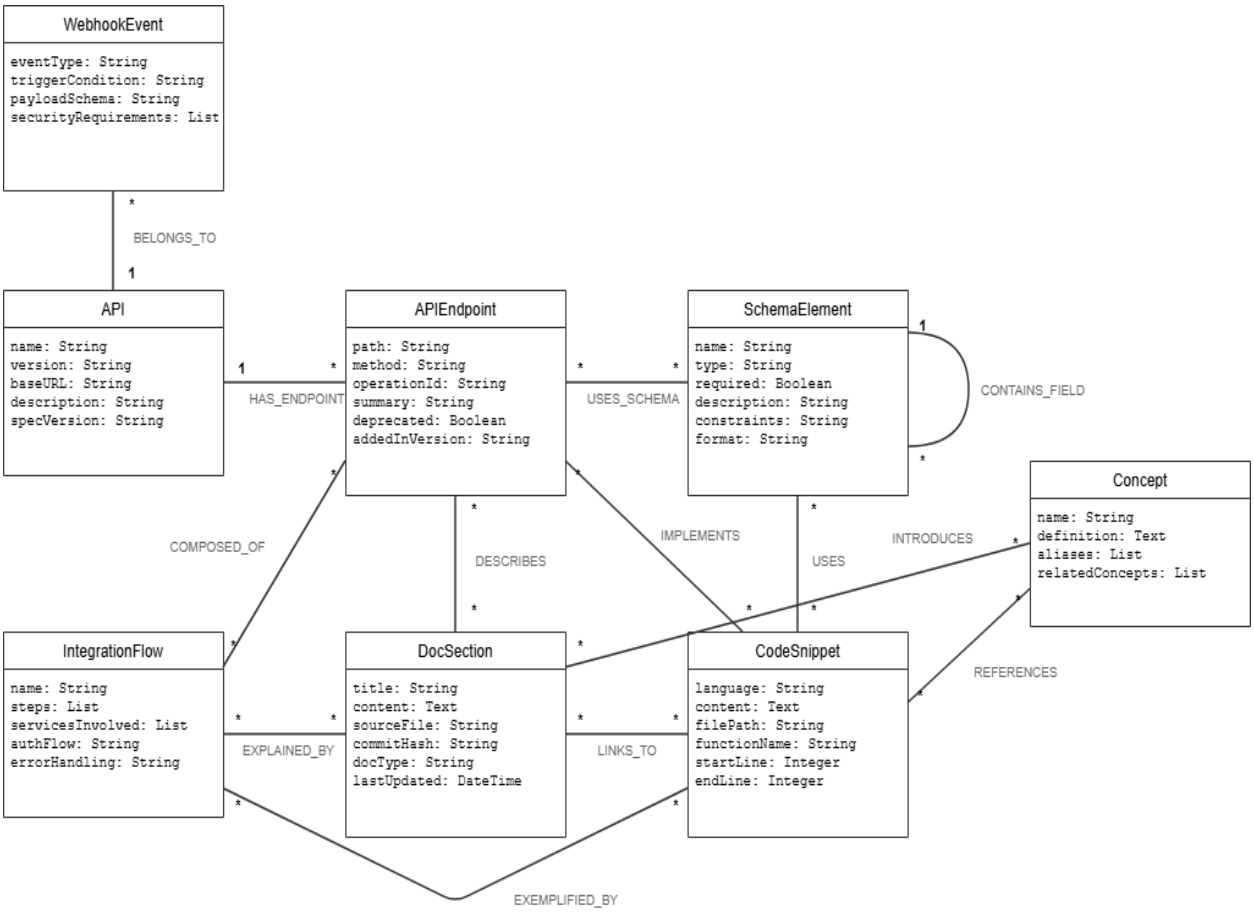

Figure 1: Entity Relationships

Relationships

The value of the UKL comes from the relationships that connect entities across sources. These relationships allow users to navigate from documentation to specs to code without searching each one separately.

Core Relationship Types

The following relationship types define how entities connect in the UKL.

API ↔ Endpoint Relationships

- **API → HAS_ENDPOINT → APIEndpoint** Establishes the structural hierarchy. Example: "Transaction Webhooks API" HAS_ENDPOINT "/balancePlatform.transaction.created"

Endpoint ↔ Schema Relationships

- **APIEndpoint → USES_SCHEMA → SchemaElement** Links endpoints to their request/response data structures. Example: The transaction webhook endpoint USES_SCHEMA "TransactionNotificationRequestV4"
- **APIEndpoint → RETURNS_SCHEMA → SchemaElement** Specifies response payloads.

Schema ↔ Schema Relationships

- **SchemaElement → CONTAINS_FIELD → SchemaElement** Represents nested object structures. Example: "Transaction" CONTAINS_FIELD "amount" (of type "Amount")
- **SchemaElement → REFERENCES → SchemaElement** Captures schema references. Example: "Transaction" REFERENCES "ResourceReference" for balanceAccount

Documentation Relationships

- **DocSection → DESCRIBES → APIEndpoint** Links documentation to the APIs it explains.
- **DocSection → INTRODUCES → Concept** Captures domain knowledge introduced in docs. Example: "Balance Platform Overview" INTRODUCES "balancePlatform" concept
- **DocSection → LINKS_TO → CodeSnippet** Connects documentation to executable examples. Example: "Making a Transfer" LINKS_TO "create_transfer.java"

Code Relationships

- **CodeSnippet → IMPLEMENTS → APIEndpoint** Shows which code demonstrates an API operation. Example: "TransferService.createTransfer()" IMPLEMENTS "/transfers POST"
- **CodeSnippet → USES → SchemaElement** Maps code to the data structures it manipulates. Example: Java snippet USES "Transaction" schema
- **CodeSnippet → REFERENCES → Concept** Links code to domain concepts. Example: Code REFERENCES "balancePlatform" when initializing client

Integration Flow Relationships

- **IntegrationFlow → COMPOSED_OF → APIEndpoint** Models end-to-end workflows. Example: "Authorization Flow" COMPOSED_OF "/authorisation"
- **IntegrationFlow → EXPLAINED_BY → DocSection** Links workflows to documentation. Example: "Transfer" EXPLAINED_BY "Transfer Integration Guide"
- **IntegrationFlow → EXEMPLIFIED_BY → CodeSnippet** Provides working code for workflows. Example: "Dispute Flow" EXEMPLIFIED_BY "dispute.py"

Relationship Cardinality

The table below defines the cardinality for each relationship type.

Relationship	Cardinality	Rationale
API → HAS_ENDPOINT	1:N	One API contains multiple endpoints
APIEndpoint → USES_SCHEMA	N:M	Endpoints can use multiple schemas; schemas can be reused
DocSection → DESCRIBES	N:M	Docs can describe multiple endpoints; endpoints can have multiple doc sections
CodeSnippet → IMPLEMENTS	N:M	One code snippet might demo multiple endpoints; one endpoint might have multiple examples
Concept → REFERENCED_BY	N:M	Concepts appear in multiple places; code/docs reference multiple concepts

Attributes and Metadata

Every entity in the UKL carries metadata for traceability, versioning and quality assessment. This metadata is used to determine where information came from and how reliable it is.

Versioning

Version information links entities to specific releases and helps track how knowledge evolves.

- API Version: Links entities to specific API releases (v4, v30)
- Doc Version: Tracks documentation updates via commit hashes
- Schema Version: Captures evolution of data structures
- Code Version: Tags code snippets with SDK/library versions

Provenance

Provenance metadata tracks the origin of each fact.

- Source Type: openapi, documentation, code
- Source Location: File path, URL, or repository reference
- Extraction Method: manual, automated, inferred
- Last Verified: Timestamp of last validation

Confidence Scores

Confidence scores measure how reliable a relationship is.

- 1.0: Fact appears identically in multiple authoritative sources (OpenAPI + Docs + Code)
- 0.8-0.9: Fact appears in primary source (OpenAPI) and one secondary source
- 0.5-0.7: Fact inferred from documentation or code patterns
- <0.5: Low confidence; requires human review

Freshness Indicators

Freshness metadata tracks how current the information is.

- Created Date: When entity was first added to UKL
- Modified Date: When entity was last updated
- Source Updated Date: When the source file changed
- Staleness Flag: Boolean indicating if entity needs review

Contextual Tags

Tags enable filtering and scoped queries.

- Environment: test, live
- Product Domain: Balance Platform, Disputes, Issuing
- Integration Type: webhook, REST API, SDK
- Audience: developer, internal, partner

Summary

The Conceptual Model gives the UKL a formal structure by defining entities and relationships. Metadata for versioning, provenance, confidence and freshness keeps the knowledge current.

System Design

The System Design defines how content from documentation, OpenAPI and code gets into the UKL. This covers the architecture, ingestion pipelines, normalization and storage.

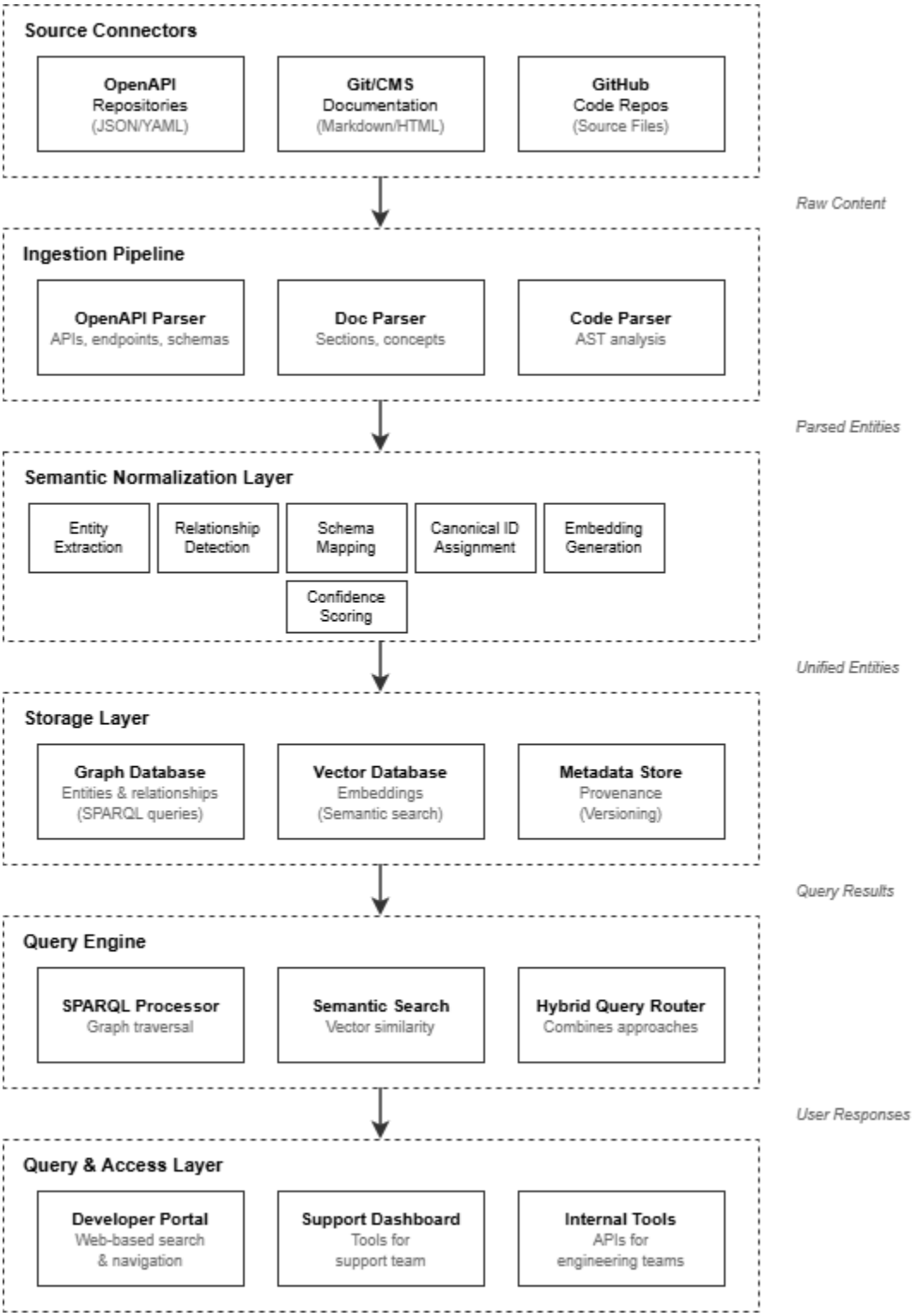

Figure 2: System Architecture

Source Connectors

Source connectors interface with external systems to retrieve content.

- OpenAPI repositories (JSON/YAML spec files)
- Git/CMS systems (Markdown documentation)
- GitHub repositories (code files)

Ingestion Pipeline

Parses and extracts structured information from each source type:

- OpenAPI Parser: Extracts APIs, endpoints, schemas
- Documentation Parser: Processes Markdown/HTML, extracts sections and concepts
- Code Parser: Uses Abstract Syntax Tree (AST) analysis to extract functions, classes, imports

Semantic Normalization Layer

The semantic normalization layer transforms raw extractions into unified semantic entities:

- Entity Extraction: Identifies core entities (APIs, endpoints, docs, code)
- Relationship Detection: Infers connections between entities
- Schema Mapping: Aligns field names and types across sources
- Canonical ID Assignment: Creates unique identifiers for deduplication
- Embedding Generation: Creates vector representations for semantic search
- Confidence Scoring: Assigns reliability scores to extractions

Storage Layer

The storage layer persists the unified knowledge in separate databases:

- Graph Database: Stores entities and relationships for SPARQL queries
- Vector Database: Stores embeddings for semantic similarity search
- Metadata Store: Tracks provenance

Query Engine

The query engine provides multiple query interfaces:

- SPARQL Processor: Handles graph traversal queries
- Semantic Search: Performs vector similarity matching
- Hybrid Query Router: Combines both approaches

Query & Access Layer

The query and access layer provides user-facing interfaces:

- Developer Portal: Web-based search and navigation

- Support Dashboard: Tools for support team
- Internal Tools: APIs for engineering teams

Summary

The architecture transforms content from three source types into a queryable knowledge graph. Each source type has its own ingestion strategy. The normalization layer creates consistency through canonical IDs, deduplication and embedding generation. The storage layer uses separate databases for each query pattern. Lifecycle management keeps the UKL in sync through change detection and versioning.

Query Mechanism

The Query Mechanism is how users access the UKL. It combines SPARQL graph traversal, vector search and hybrid retrieval.

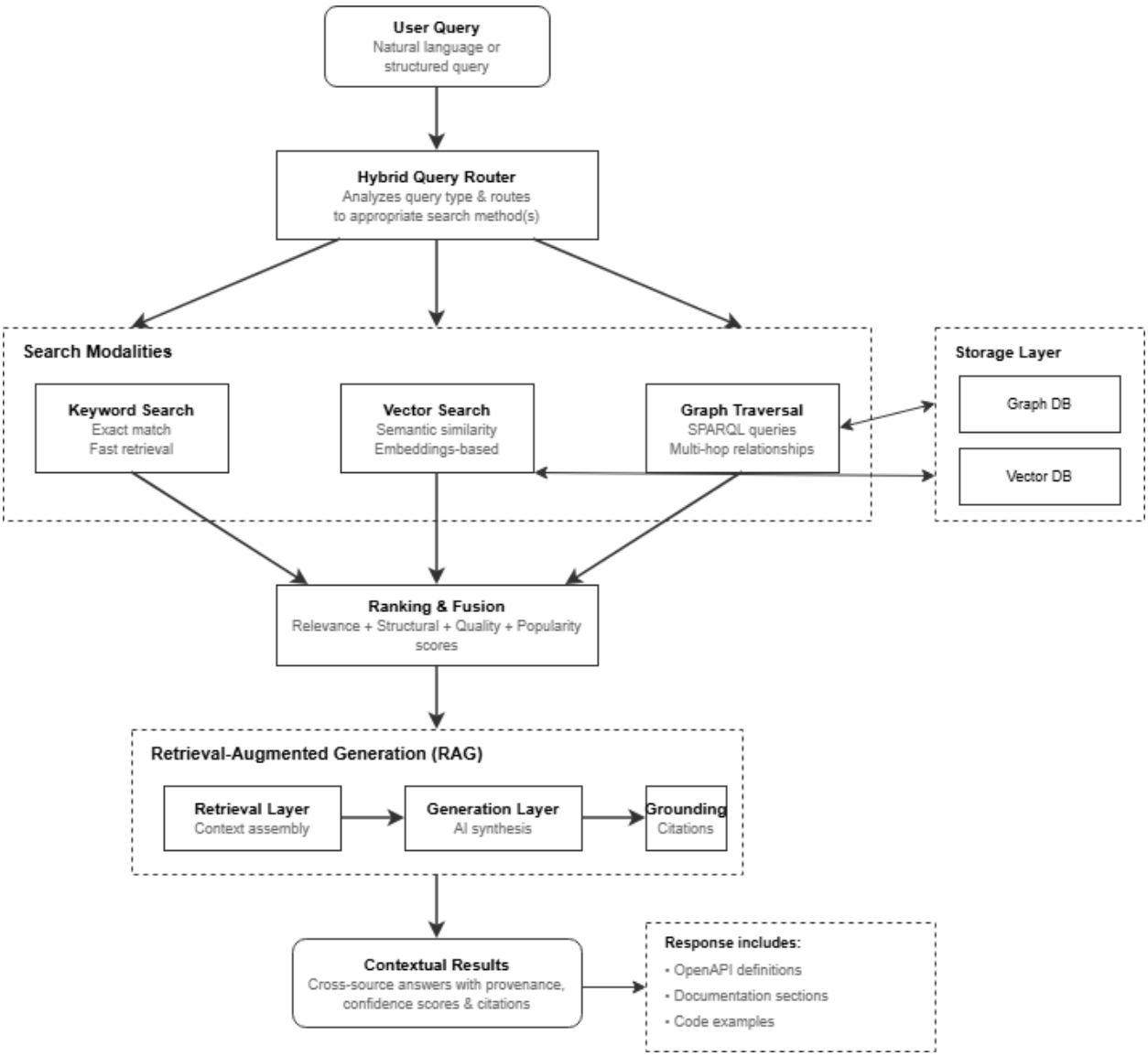

Figure 3: Query flow

Unified Search Experience

The UKL has a single search interface across documentation, OpenAPI and code.

Search Capabilities

Natural Language Queries

Users can ask questions and the system interprets intent, extracts entities and returns grounded answers.

Examples:

- "How do I create a payment session?"
- "What changed in the Transaction webhook between v3 and v4?"
- "Show me code examples for handling dispute defense documents"

- "What fields are required for balance account transfers?"

Structured Queries

Users can create queries for results, validation, analytics and reporting.

Examples:

- "List all endpoints added in API version v4"
- "Find all documentation sections that have not been updated in 6 months"
- "Show code snippets that reference deprecated schemas"

Contextual Results

Query results include provenance metadata showing where the information came from, confidence scores and last update timestamps.

Cross-Source Integration

A single query can return results from all three knowledge sources:

- The OpenAPI endpoint definition
- Documentation explaining how to implement it
- Java, Python, and Node.js code examples
- Related concepts like balanceAccount and paymentInstrument

Retrieval-Augmented Generation (RAG)

AI models are layered on the UKL to generate answers that go beyond retrieval.

RAG Architecture

Retrieval Layer

The retrieval layer pulls nodes and relationships from the knowledge graph and vector index so the AI model has current information.

Process:

1. "How do I handle relayed authorization for issuing cards?"
2. Entity extraction: Identifies relayed authorization, issuing cards, authorization webhook
3. Graph traversal: Retrieves related endpoints, schemas, docs, code
4. Vector search: Finds semantically similar content
5. Context assembly: Combines structured and unstructured knowledge

Generation Layer

The generation layer uses AI models to process the retrieved content.

Grounding

Every generated answer includes references to documentation, OpenAPI specs or code.

Hybrid Search (Semantic + Symbolic)

The UKL combines three different search methods.

Search Modalities

Keyword Search

Fast retrieval for exact matches. Used when a user knows specific terms.

Example: Searching for "balancePlatform.transaction.created" returns the exact webhook endpoint.

Vector Search

Uses similarity search for semantic queries.

Example: Searching for "money transfer flow" semantically matches docs about "balance account transfers" and "payment processing."

Graph Traversal

Graph traversal supports symbolic queries across relationships and allows for multi hop queries.

Example: "Which documentation sections explain the endpoints that use the Transaction schema?" requires traversing:

SchemaElement("Transaction") → USED_BY → APIEndpoint → DESCRIBED_BY → DocSection

Hybrid Query Routing

The UKL determines which search method to use based on query structure.

Query Type	Primary Method	Secondary Method	Example
Exact match	Keyword	Vector	"POST /transfers endpoint"
Conceptual	Vector	Graph	"How to handle payment failures?"
Relational	Graph	Vector	"What code implements this endpoint?"
Exploratory	Vector	Graph + Keyword	"Best practices for webhook security"

Ranking Results

Results from different search methods are combined and reranked based on:

1. Relevance Score: Semantic similarity (vector search)
2. Structural Score: Relationship strength (graph traversal)
3. Quality Score: Confidence + freshness metadata
4. Popularity Score: Usage metrics (if available)

SPARQL Query Examples

The following SPARQL queries show how the UKL can be traversed to answer specific questions.

Example 1: Find All Endpoints for a Specific API

Use Case: A user wants to see all available endpoints in the Transaction Webhooks API.

```
PREFIX ukl: <http://examplecorp.com/ukl#>
SELECT ?endpoint ?path ?method ?summary
WHERE {
  ?api a ukl:API ;
```

```
      ukl:name "Transaction Webhooks API" ;
      ukl:HAS_ENDPOINT ?endpoint .
    ?endpoint ukl:path ?path ;
      ukl:method ?method ;
      ukl:summary ?summary .
  }
  ORDER BY ?path
```

Example 2: Get Documentation and Code Examples for an Endpoint

Use Case: A user found an endpoint and wants related documentation and code snippets.

```
PREFIX ukl: <http://examplecorp.com/ukl#>
SELECT ?docTitle ?snippetLanguage ?snippetPath
WHERE {
  ?endpoint a ukl:APIEndpoint ;
    ukl:path "/transfers" ;
    ukl:method "POST" .
  OPTIONAL {
    ?doc ukl:DESCRIBES ?endpoint ;
      ukl:title ?docTitle .
  }
  OPTIONAL {
    ?snippet ukl:IMPLEMENTS ?endpoint ;
      ukl:language ?snippetLanguage ;
      ukl:filePath ?snippetPath .
  }
}
```

Example 3: Identify Stale Documentation

Use Case: A user wants to find docs that have not been updated since the related API endpoint changed.

```
PREFIX ukl: <http://examplecorp.com/ukl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT ?docTitle ?docUpdated ?endpointPath ?endpointUpdated
WHERE {
  ?doc a ukl:DocSection ;
    ukl:DESCRIBES ?endpoint ;
    ukl:title ?docTitle ;
    ukl:lastUpdated ?docUpdated .
  ?endpoint a ukl:APIEndpoint ;
    ukl:path ?endpointPath ;
    ukl:lastUpdated ?endpointUpdated .
  FILTER (?endpointUpdated > ?docUpdated)
}
ORDER BY DESC(?endpointUpdated)
```

Validation

Data quality is important for the UKL to be able to return valid responses. The validation framework has automated checks, cross-source validation, confidence scoring, human review and monitoring drift.

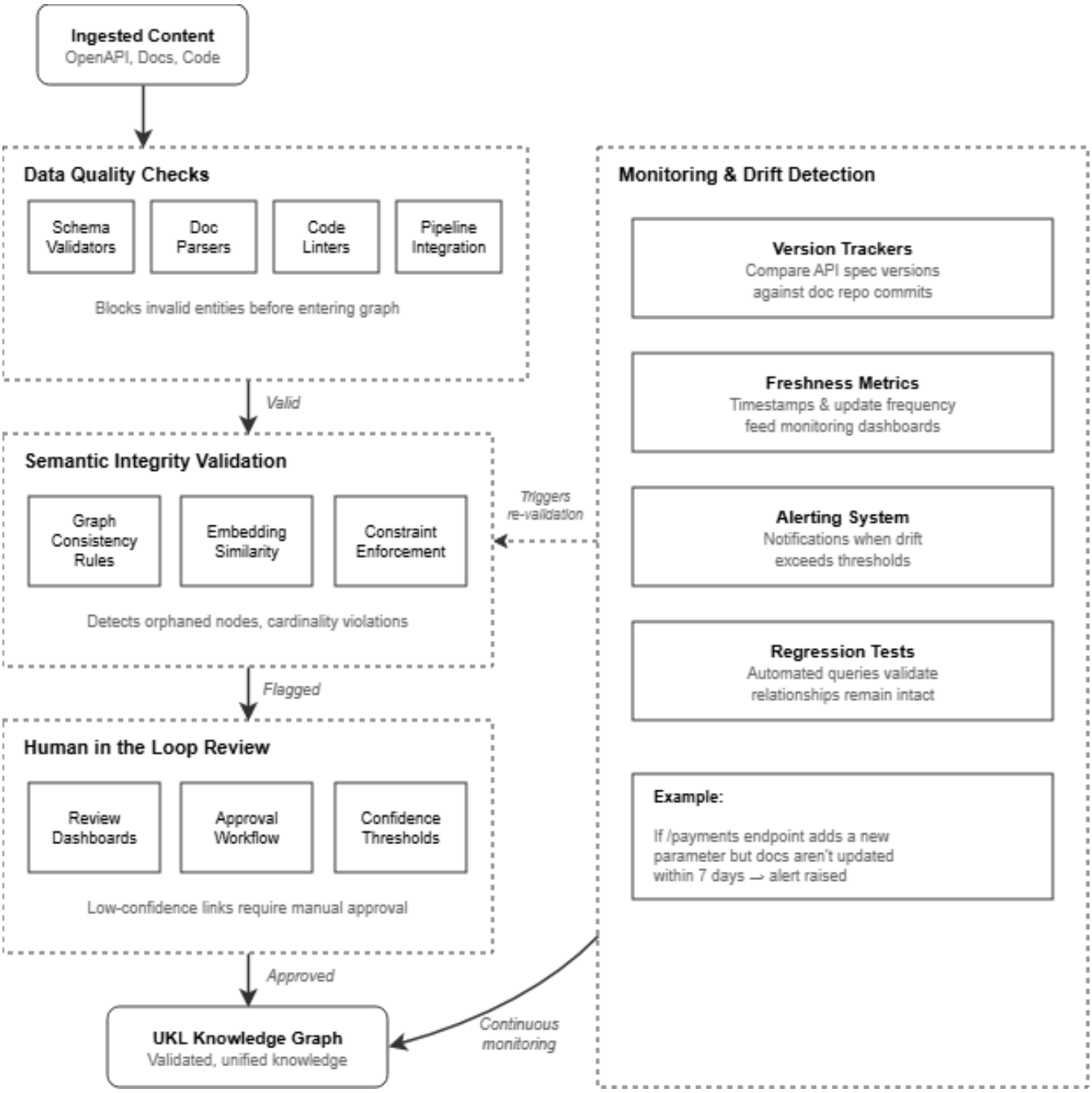

Figure 4: Data validation workflow

Data Quality Checks

The following automated checks validate content as it is ingested:

- Schema Validators: Automated tools compare ingested schema elements against the specs.
- Doc Parsers: NLP models extract field names from Markdown docs and crosscheck them against spec definitions.

- Code Linters: Static analysis ensures code snippets reference valid endpoints and fields.
- Pipeline Integration: These checks run during ingestion, blocking invalid entities before they enter the graph.

As an example, if a code snippet references balancePlatform but no doc section introduces that concept, the system flags the missing documentation and suggests linking or authoring a new section.

Semantic Integrity Validation

The UKL validates semantic integrity to confirm relationships between entities are consistent:

- Graph Consistency Rules: SPARQL queries detect orphaned nodes (endpoints with no doc section).
- Embedding Similarity: Vector comparisons ensure that concepts referenced in docs align semantically with schema and code.
- Constraint Enforcement: Relationships must meet defined cardinality (every endpoint must be EXPLAINED_BY at least one DocSection).

Human in the Loop Review

Some decisions require human review. The UKL has workflows for human review of flagged items:

- Review Dashboards: Flagged inconsistencies appear in a UI for doc teams and API owners.
- Approval Workflow: Changes require sign-off before propagation.
- Confidence Thresholds: Low-confidence links are routed to human review.

As an example, when the system auto-links a Java SDK snippet to the /sessions endpoint with 70% confidence, it routes the link to the API owner for manual approval.

Monitoring & Drift Detection

Over time APIs change and documentation will fall out of sync. The UKL has monitoring to detect this drift.

- Version Trackers: Compare API spec versions against doc repo commits.
- Freshness Metrics: Timestamps and update frequency feed dashboards.
- Alerting System: Notifications sent to doc owners when drift exceeds thresholds.
- Regression Tests: Automated queries validate that previously correct relationships remain intact after updates.

As an example, if the /payments endpoint is updated in the spec to include a new parameter, but the docs have not been updated within 7 days, the system raises an alert and shows the discrepancy in a freshness dashboard.

Summary

The validation strategy ensures accuracy as content is ingested and as sources change. Automated checks catch errors at ingestion before they enter the graph. Semantic integrity validation checks that relationships are consistent. Human review handles cases with low confidence. Monitoring detects drift between sources and alerts teams when they fall out of sync.

Without this framework the UKL would degrade over time as sources change. Automated checks combined with human review and monitoring is how the UKL maintains reliablity.

Conclusion

The UKL addresses knowledge being spread across sources that are not connected. Unifying these sources reduces the time spent finding valid information.

The Conceptual Model represents knowledge as connected entities. The System Design defines how content flows into the graph. The Query Mechanism combines symbolic and semantic search. The Validation Strategy maintains data quality through automated checks, human review and monitoring.

These components create a foundation for knowledge that changes with the platform while staying accurate.